

Family Album

Setup and operations guide for the self-hosted trip photo album

This guide explains what the Family Album application is, how to install and run it with Docker, how to configure email, storage and maps, how to bring family members on board, and how to use the day-to-day features such as trips, photo uploads, GPS track imports, themes and sharing. It ends with a development setup, backup advice and troubleshooting.

Contents

- 1. What the application does
- 2. Requirements
- 3. Quick start with Docker
- 4. Configuration reference (.env)
- 5. Signing in and inviting family
- 6. Using the album: trips, uploads, activities, tracks, themes, sharing
- 7. Running behind a reverse proxy
- 8. Backups and upgrades
- 9. Development setup and tests
- 10. Troubleshooting

1. What the application does

Family Album is a private web app for organising family trip photos. Photos belong to a **trip**, and within a trip to an **activity** (a hike, a bike ride, a boat tour, a meal). Everything can be browsed on a **timeline** grouped by day and on a **map**. GPS tracks from a watch, bike computer or Google location history give activities a route and fitness statistics, and can place photos that have no GPS data of their own.

- **Automatic organisation.** Uploaded photos land on the right trip by the date they were taken, and on the right activity by time of day.
- **Any camera or phone.** JPEG, PNG, WebP and iPhone HEIC are accepted. Originals are kept; web-sized WebP versions are generated for display.
- **Correct times.** Camera clocks are reconciled using the EXIF time-zone offset, the GPS position, or the trip's time zone. A one-click fix handles cameras left in the wrong zone.
- **Tracks and stats.** Import GPX, Garmin FIT, or Google Timeline exports. Activities show distance, moving time, elevation, pace or speed, heart rate, cadence, power and calories, with charts linked to the map.
- **Photos placed from tracks.** A photo taken during a hike without GPS is positioned by interpolating along the track.

- **Themes.** Each trip gets a look: lighthouse coast, Scottish highlands, Everglades swamp, desert canyon, alpine, or classic.
- **Sharing.** Trips are private by default, can be shared by a secret link, or made public.
- **Photo links.** Photos can be marked as the same scene, before/after, panorama parts, or related.
- **Passwordless sign-in.** Family members sign in with an emailed link. An admin invites them.

How it is built

The app is a single Next.js server with a PostgreSQL database and a folder for photos. Background work (processing uploads, parsing track files, geotagging) runs in a job queue stored in Postgres, so there is nothing else to run.

Component	Technology	Role
Web app	Next.js 16, React 19, TypeScript	Pages, uploads, API routes
Database	PostgreSQL 16 with Prisma 7	Trips, activities, photos, tracks, users
Photo storage	Local folder (Docker volume)	Originals plus generated thumbnails
Jobs	pg-boss (in the Postgres database)	Photo processing, track imports, geotagging
Images	sharp, exifr, heic-convert	EXIF reading, resizing, HEIC conversion
Maps	MapLibre GL with OpenStreetMap tiles	Clustered photo markers and track lines
Email	Nodemailer over SMTP	Sign-in and invite links

2. Requirements

- A machine that runs **Docker** and **Docker Compose v2**: a home server, NAS, Raspberry Pi 4 or better, or a small VPS. 2 GB of RAM is comfortable; HEIC conversion is the heaviest task.
- Disk space for photos. Originals are kept, so budget for the size of your library plus roughly 15% for renditions.
- An **SMTP account** for sending sign-in emails (any provider: Gmail app password, Fastmail, Mailgun, Postmark, your ISP). Optional for testing; links can be read from the log instead.
- Optional: a domain name and a reverse proxy with HTTPS if you want to reach the album from outside your home network.

3. Quick start with Docker

- 1 Get the code onto the server, either by cloning the repository or by copying the project folder.
- 2 Create your configuration file from the template and edit it. At minimum set **ADMIN_EMAIL** to your own address.
- 3 Build and start the stack. The first build takes a few minutes.
- 4 Open the site, enter the admin email, and follow the sign-in link.

```
git clone https://github.com/Macrophage87/PhotoAlbum.git
```

```
cd PhotoAlbum
cp .env.example .env
nano .env          # set ADMIN_EMAIL, SMTP_* for real email, a new POSTGRES_PASSWORD
docker compose up --build -d
```

The stack has two services. **db** is PostgreSQL 16 with its data in the **pgdata** volume. **app** is the web server, with photos in the **photos** volume mounted at `/data/photos`. On every start the app applies pending database migrations before serving, so upgrades need no manual database step.

Browse to **`http://<server>:3000`**. Change the host port with `APP_PORT` in `.env` if 3000 is taken.

Reading the sign-in link without email

If `SMTP_HOST` is left empty, sign-in and invite links are printed to the container log instead of being sent. This is fine for first setup and for a purely local install.

```
docker compose logs -f app | grep "auth/verify"
```

Loading demo data

A seed script creates a sample lighthouse-themed trip in Acadia, Maine with a hike, its track and stats, and three sample photos, one of which is positioned from the track. It is a quick way to see every feature working.

```
docker compose exec app node_modules/.bin/tsx prisma/seed.ts
```

Useful commands

```
docker compose ps          # service status
docker compose logs -f app  # follow app log
docker compose restart app  # restart the web app
docker compose down        # stop (volumes are kept)
docker compose pull && docker compose up --build -d  # rebuild after updating the code
```

4. Configuration reference (.env)

All settings live in the `.env` file next to `docker-compose.yml`. The compose file also reads `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_DB` and `APP_PORT` from it. `DATABASE_URL` and `PHOTO_STORAGE_ROOT` are set by the compose file for the container, so you only need to change them for a non-Docker install.

Variable	Default	Purpose
<code>APP_URL</code>	<code>http://localhost:3000</code>	Public URL of the site. Used in every emailed link and in redirects. Set it to your real address (with https) when behind a proxy.
<code>ADMIN_EMAIL</code>	<code>you@example.com</code>	The first person to sign in with this address becomes an admin. Also allowed to sign in before any invites exist.
<code>SMTP_HOST</code>	(empty)	Mail server hostname. Leave empty to log links instead of sending mail.
<code>SMTP_PORT</code>	587	587 for STARTTLS, 465 for implicit TLS.
<code>SMTP_USER / SMTP_PASS</code>	(empty)	Mail server credentials.

Variable	Default	Purpose
SMTP_SECURE	false	Set true when using port 465.
SMTP_FROM	Family Album <album@example.com>	Sender shown in emails. Many providers require it to match the account.
POSTGRES_USER / _PASSWORD / _DB	photoalbum	Database credentials used by both containers. Change the password on an internet-facing host.
APP_PORT	3000	Host port published by Docker.
STORAGE_DRIVER	local	Storage backend. Only local is implemented; the code has an interface for adding S3 later.
PHOTO_STORAGE_ROOT	/data/photos	Where originals and renditions are written (inside the container).
MAX_UPLOAD_BYTES	104857600 (100 MB)	Largest single photo accepted.
MAX_IMPORT_BYTES	2147483648 (2 GB)	Largest track or Google export file accepted.
RUN_WORKER	true	Run background jobs inside the web process. Set false and start the worker container (docker compose --profile worker up -d) to split them out.
NEXT_PUBLIC_TILE_URL	(empty, uses OpenStreetMap)	Raster tile template for the map, e.g. from MapTiler or Stadia.
NEXT_PUBLIC_MAP_STYLE_URL	(empty)	Full MapLibre style JSON URL. Overrides the tile URL.
NEXT_PUBLIC_MAP_GLYPHS_URL	(empty)	Font glyph URL template for map labels. All NEXT_PUBLIC values are compiled into the browser bundle: rebuild with docker compose up --build after changing them.

Gmail example. SMTP_HOST=smtp.gmail.com, SMTP_PORT=587, SMTP_SECURE=false, SMTP_USER=your address, SMTP_PASS=an App Password generated in your Google account security settings (normal passwords are rejected), SMTP_FROM=the same address. Restart the app after changing .env: docker compose up -d.

5. Signing in and inviting family

There are no passwords. A person types their email on the sign-in page and receives a link that is valid for 15 minutes and can be used once. The resulting session lasts 90 days on that browser.

Who can sign in

- The address in ADMIN_EMAIL, always. The first sign-in with it creates the admin account.
- Anyone who already has an account.
- Anyone with a pending invite for their address.
- Everyone else sees the same 'check your email' message but no email is sent and no account is created, so the site never reveals who is a member.

Inviting members

- 1 Sign in as the admin and open **Admin** from the top navigation (the /admin page).
- 2 Enter the relative's email address and choose Member or Admin. An invite email is sent (or logged when SMTP is not set up).
- 3 The relative opens the invite link, which sends them a sign-in link for that address. From then on they simply sign in with their email.
- 4 The Admin page lists members and pending invites. From there you can change roles, revoke an invite, or remove a member.

Members can create trips, upload photos, edit anything, and share trips. Admins can additionally manage members. Every signed-in member sees every trip; the album is a shared family space, not per-user galleries.

6. Using the album

Creating a trip

Choose **New trip** on the front page. Give it a title, start and end dates, the local time zone of the destination, and a theme. The date range matters: it is how uploaded photos are matched to the trip, and how Google location exports are filtered. The time zone is used when a photo's camera did not record one and when grouping days on the timeline.

Uploading photos

- Use the **Upload** button inside a trip's Photos tab, or the global Upload page. Drag and drop or pick files; several upload at once.
- Each photo is processed in the background: EXIF date, GPS and camera details are read, the image is rotated correctly, and thumbnail and medium WebP renditions are made. Tiles show a spinner until ready.
- A photo uploaded from a trip page is attached to that trip. Uploaded from the global page, it is matched to whichever trip's dates contain the day it was taken. If several trips overlap, it stays unassigned and shows a 'needs trip' badge until you assign it.
- If the photo's time falls inside an activity's window on that trip, it is attached to the activity too.
- The photo detail page shows EXIF data, lets you edit the caption, move the photo to a trip or activity, shift the time zone if the camera was set wrong, set it as the trip cover, and link it to other photos.
- In the gallery, select several photos for bulk actions: assign to an activity, move to another trip, or delete.

iPhone HEIC photos work, but conversion is slow (a few seconds each). If you prefer, set the iPhone camera to 'Most Compatible' so it saves JPEG. Renditions are always WebP, so browsers never need HEIC support.

Activities

An activity is anything you did within a trip: hike, bike, run, kayak, drive, boat, sightseeing, food, or other. Create one by hand from the trip's Activities tab with a title, type and time window, or let a FIT or GPX import create it. Photos whose time falls inside the window are attached automatically and shown inside the activity's card on the timeline.

Importing GPS tracks

Open the trip's **Import** page and upload a file. The import runs in the background and reports what it created.

Source	What to upload	Result
Garmin, Wahoo, Strava, Komoot, AllTrails, phone apps	.gpx or .fit file (export the activity from the app or Garmin Connect)	An activity with route, statistics and charts. Sport is detected from FIT files. Photos in its time window are attached and, if they lack GPS, positioned from the track.
Google Maps Timeline (on the phone)	Google Maps > profile picture > Your Timeline > (three dots) > Location and privacy settings > Export Timeline data. Upload the resulting Timeline.json.	One location trace per day of the trip, drawn dashed on the map. No activity is created.
Google Takeout (older exports)	Records.json, or the monthly Semantic Location History JSON files.	Same as above.

Only points inside the trip's dates are imported, so uploading a whole multi-year export is safe; the app streams the file rather than loading it into memory. Statistics computed from a track: distance, moving and elapsed time, elevation gain and loss, average and maximum speed or pace, heart rate, cadence, power and normalized power, and calories. Values recorded by a Garmin device in the FIT file take precedence over computed ones so they match what the watch showed.

Timeline and map

- The trip **Timeline** groups photos and activities by local day, with a day list on the side for jumping around. Photos taken during an activity appear inside its card next to a small route drawing and key stats.
- The trip **Map** shows clustered photo markers and colour-coded track lines. Click a cluster to zoom in, a marker to preview and open the photo, a track to open the activity.
- The **activity page** shows the full-size map, the stat grid, and elevation, speed, heart-rate and power charts. Moving the mouse along a chart moves a marker along the route.
- The global **Map** and **Timeline** pages in the top navigation show every trip together, each keeping its own theme colours.

Themes

Pick a theme when creating a trip or later in its Settings. A theme sets the palette, fonts, illustrated header art, map marker shape and decorative motif for that trip's pages, including the shared and public views.

Theme	Feel	Suggested for
Classic	Neutral slate and blue	Anything

Theme	Feel	Suggested for
Lighthouse Coast	Navy, fog white, red and white stripes, sea glass	Maine, coastal New England, the Pacific coast
Highlands	Heather purple, peat brown, moss green, mist	Scotland, Ireland, moorland
Swamp	Deep green, cypress brown, egret white, sunset orange	Everglades, bayou, wetlands
Desert Canyon	Terracotta, sandstone, sage, turquoise sky	Utah, Arizona, the Southwest
Alpine	Ice blue, granite grey, pine green, snow	The Alps, Rockies, ski trips

Sharing a trip

Each trip has a visibility setting in its Settings page.

- **Private** (default): only signed-in family members can see it.
- **Anyone with the link**: a secret link is generated. Relatives without accounts can view the photos, activities, timeline and map, but cannot edit. The link can be rotated at any time, which immediately disables the old one.
- **Public**: anyone can browse the trip without signing in and it appears on the front page for visitors.

Privacy. A shared or public trip exposes every photo, activity and location trace on that trip, which reveals where people were and when. Keep photos you would not share on a private trip, and rotate a link if it spreads further than intended.

7. Running behind a reverse proxy

To reach the album from outside your network, put it behind a reverse proxy that terminates HTTPS (Caddy, nginx, Traefik, or your NAS's built-in proxy). Two things need attention:

- Set **APP_URL** in `.env` to the public address, for example `https://album.example.com`, so emailed links and redirects use it.
- Raise the proxy's request body limit. Photos are up to 100 MB by default and Google exports can be far larger. The app-side caps are `MAX_UPLOAD_BYTES` and `MAX_IMPORT_BYTES`.

Caddy example (automatic HTTPS):

```
album.example.com {
    reverse_proxy localhost:3000
    request_body {
        max_size 2GB
    }
}
```

nginx example:

```
server {
    listen 443 ssl;
    server_name album.example.com;
    client_max_body_size 2g;
```

```

    proxy_read_timeout 600s;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}

```

When exposed to the internet, also change `POSTGRES_PASSWORD` from the default, and consider not publishing the database port at all (the default compose file does not).

8. Backups and upgrades

Two things hold all your data: the **pgdata** volume (database) and the **photos** volume (originals and renditions). Back up both. The photos volume is the irreplaceable one.

```

# database dump
docker compose exec db pg_dump -U photoalbum photoalbum | gzip > album-db-$(date +%F).sql.gz

# photos (copy the volume contents)
docker run --rm -v photoalbum_photos:/data -v $(pwd):/backup alpine \
    tar czf /backup/album-photos-$(date +%F).tgz -C /data .

```

The volume name is prefixed with the compose project name, normally the folder name; check with `docker volume ls`. Restoring is the reverse: create the volumes, extract the archive into the photos volume, and pipe the dump into `psql` on the db container.

Upgrading

```

git pull
docker compose up --build -d

```

Database migrations run automatically at container start. Take a database dump before upgrading, as a precaution.

9. Development setup and tests

For working on the code, run the app on your machine with Node 22 and `pnpm`, and use Docker only for Postgres and a local mail catcher.

```

pnpm install
cp .env.example .env          # DATABASE_URL should point at localhost:5432
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d db mailpit
pnpm prisma migrate dev       # create the schema
pnpm db:seed                  # optional demo data
pnpm dev                       # http://localhost:3000

```

Mailpit catches all outgoing mail. Set `SMTP_HOST=localhost` and `SMTP_PORT=1025` in `.env` and read sign-in emails at `http://localhost:8025`. Without SMTP settings the links are printed in the terminal running `pnpm dev`.

Scripts

Command	What it does
<code>pnpm dev / pnpm build / pnpm start</code>	Development server / production build / run the build
<code>pnpm lint / pnpm typecheck</code>	ESLint and TypeScript checks
<code>pnpm test</code>	Vitest unit tests (parsers, stats, timeline, auth). Migrates, and creates if needed, a <code>photoalbum_test</code> database before running.
<code>pnpm build && pnpm test:e2e</code>	Playwright smoke tests (sign-in, upload, sharing) against a <code>photoalbum_e2e</code> database
<code>pnpm db:migrate / pnpm db:deploy / pnpm db:seed</code>	Create a migration / apply migrations / load demo data
<code>pnpm worker</code>	Run the background job worker as its own process (with <code>RUN_WORKER=false</code> on the web process). In Docker use the worker compose profile instead.
<code>node scripts/make-fixtures.mjs</code>	Regenerate test fixture files (GPX, FIT, Google JSON, photos)

Code layout

<code>src/app</code>	routes (Next.js App Router), including <code>/api</code> handlers
<code>src/components</code>	UI components (photos, timeline, map, charts, admin)
<code>src/lib/auth</code>	magic links, sessions, trip access rules
<code>src/lib/images</code>	EXIF, renditions, HEIC conversion
<code>src/lib/tracks</code>	GPX / FIT / Google parsers, stats, simplification, storage encoding
<code>src/lib/jobs</code>	pg-boss queue and the <code>process-photo / import-track / geotag</code> jobs
<code>src/themes</code>	theme registry and per-theme art
<code>prisma</code>	schema, migrations and seed
<code>tests</code>	unit tests, fixtures, Playwright specs

A GitHub Actions workflow runs lint, typecheck, unit tests and a Docker build on every push.

10. Troubleshooting

Symptom	Likely cause and fix
Sign-in says 'check your email' but nothing arrives for a new person	Only <code>ADMIN_EMAIL</code> , existing members and invited addresses receive a link; everyone else gets the same message and no email. Invite them from the Admin page, and check <code>.env</code> spelling (restart the app after edits).
No sign-in email arrives	<code>SMTP_HOST</code> empty (links are in the log: <code>docker compose logs app grep auth/verify</code>), wrong port/SECURE combination, or the provider rejects <code>SMTP_FROM</code> . Test with Mailpit first.
Links in emails point to localhost	<code>APP_URL</code> is still the default. Set it to the public URL and restart.
Uploads fail around 1 MB or 100 MB	Reverse proxy body limit (raise <code>client_max_body_size / max_size</code>) or <code>MAX_UPLOAD_BYTES</code> .
Photo stays on the spinner	Processing job failed; see the app log. A failed photo shows its error on the detail page. HEIC files take several seconds each.
Photo landed on the wrong day or trip	Camera clock or time zone. Open the photo and use the time-zone shift control, or set the trip's time zone correctly before uploading.

Symptom	Likely cause and fix
Photo has no location on the map	No GPS in the file and no track covering that time. Import a GPX/FIT/Google trace for that day; photos are then geotagged from it automatically.
Google import created nothing	No points inside the trip's dates. Check the trip dates and that the file is a Timeline export (Timeline.json, Records.json or Semantic Location History).
Map is blank	Tile server unreachable or NEXT_PUBLIC_TILE_URL / NEXT_PUBLIC_MAP_STYLE_URL wrong. Clear them to fall back to OpenStreetMap. NEXT_PUBLIC values are baked in at build time, so rebuild after changing them.
Container restarts on start	Migration failure or database not ready; run docker compose logs app. The db service has a health check so app waits for it.
Disk full	Photos volume. Move Docker's data root or mount a larger disk at the volume; originals are never deleted automatically.